

DG Interns Hub

Cybersecurity Internship

Week 6 Assignment Report

Web Security & Vulnerability Testing (VAPT Basics)

Submitted by: Ajith Kumar

Batch / Intern ID: DG/AUGUST/CYBER/031

Date: 30-09-2026

Table of Contents

Introduction to VAPT 2

Task 1: Study OWASP Top 10..... 3

 1) SQL Injection..... 3

 2) XSS – Cross-Site Scripting..... 4

 3) Broken Authentication 5

 4) Security Misconfiguration 6

Task 2: Install & Setup Tools 7

 1) Install DVWA dependencies..... 7

 1.1 Download DVWA 8

 2)Install OWASP ZAP 9

Task 3: Perform Testing (DVWA)..... 9

 3.1 SQL Injection Testing..... 10

 3.2 Reflected XSS Testing..... 12

 3.3 Stored XSS Testing 14

 Vulnerability Findings and Risk Levels 16

Task 4: Scan Using OWASP ZAP 16

 Conclusion & Learning..... 19

Introduction to VAPT

VAPT stands for **Vulnerability Assessment and Penetration Testing**. It is a security process used to find weaknesses in systems, websites, and applications and check whether attackers can exploit them.

What is VAPT?

VAPT combines **finding security vulnerabilities** with **safely testing those vulnerabilities**. It helps organizations identify and fix security problems before real attackers can use them.

Vulnerability Assessment (VA)

VA is the process of **scanning and identifying vulnerabilities** in a system. It mainly tells us **what security weaknesses exist** and how serious they are.

Penetration Testing (PT)

PT is the process of **actively testing vulnerabilities** to see whether they can actually be exploited. It simulates real-world attacks in an authorized and controlled way.

Difference between VA and PT

- **VA:** Finds and lists vulnerabilities.
- **PT:** Tries to exploit vulnerabilities to prove their impact.

- In simple words: **VA = Find the weakness; PT = Test the weakness.**

Importance of Web-Security Testing

Web-security testing helps protect websites and applications from **hackers, data theft, unauthorized access, and other attacks**. It also helps organizations fix security issues before they cause damage.

Task 1: Study OWASP Top 10

OWASP Top 10 is a list of common and important **web application security risks**. It helps developers and security testers understand common vulnerabilities and how to prevent them.

1) SQL Injection

SQL Injection is a vulnerability where an attacker inserts malicious SQL commands into an application's input fields. If the application does not properly validate the input, the database may execute the attacker's commands.

Simple Example

Suppose a login page asks for: Username and Password

The application creates a database query using the entered values. If user input is directly added to the SQL query without proper protection, an attacker may manipulate the query's logic.

Basic idea:

User Input → Application → SQL Query → Database

If the input is not handled safely, the attacker may change what the SQL query does.

How SQL Injection Happens

SQL Injection commonly occurs when:

1. An application accepts user input.
2. The input is directly combined with an SQL query.
3. The application sends the resulting query to the database.
4. The database interprets the malicious input as SQL code.

Types of SQL Injection

- **In-band SQLi:** The attacker receives results through the same application.
- **Blind SQLi:** The application does not directly show database results, so the attacker observes differences in application behavior.
- **Out-of-band SQLi:** Data is obtained through a different communication channel.

Impact

SQL Injection can potentially allow an attacker to:

- Access unauthorized database information.

- Modify or delete data.
- Bypass application restrictions.
- Access sensitive user information.
- In serious cases, compromise other parts of the application or infrastructure.

Prevention

The main prevention techniques are:

- Use **parameterized queries / prepared statements**.
- Validate and sanitize input where appropriate.
- Give database accounts only the permissions they need.
- Avoid constructing SQL queries by directly concatenating user input.
- Regularly test applications for SQL Injection.

2) XSS – Cross-Site Scripting

Cross-Site Scripting (XSS) is a web security vulnerability where untrusted content is interpreted by a user's browser as executable script.

In simple words:

Attacker-controlled input → Web application → Victim's browser → Script executes

Simple Example

Imagine a website allows users to submit comments. If the website displays comments without properly handling untrusted content, an attacker could submit content containing a script.

When another user views the affected page, the browser may execute the script.

Types of XSS

1. Stored XSS

The malicious content is **stored on the server**, such as in a database or comment section.

When other users open the affected page, the content is delivered to their browsers.

2. Reflected XSS

The malicious input is immediately returned by the web application, often through a URL or form submission.

The victim must generally interact with the specially crafted request for the attack to occur.

3. DOM-Based XSS

The vulnerability occurs because client-side JavaScript processes untrusted data and places it into the webpage in an unsafe way.

Impact

XSS can potentially be used to:

- Perform actions in a victim's browser.
- Steal sensitive information accessible to scripts.
- Modify webpage content.
- Trick users into interacting with malicious content.

- Perform actions using the victim's existing session.

Prevention

- Properly **encode output** according to its context.
- Validate input where appropriate.
- Avoid unsafe DOM APIs and coding patterns.
- Use frameworks' built-in escaping mechanisms correctly.
- Implement an appropriate **Content Security Policy (CSP)**.
- Use secure cookie settings such as **HttpOnly** where appropriate.

3) Broken Authentication

Broken Authentication refers to weaknesses in login, session management, password handling, or account recovery mechanisms that can allow unauthorized users to access accounts.

In modern OWASP terminology, authentication-related weaknesses are largely covered under **Identification and Authentication Failures**.

Simple Example

Consider a website where a user logs in successfully. The website gives the user a session identifier.

If the application handles this session insecurely, an attacker who obtains the session identifier may potentially access the user's account.

Common Causes

- Weak password policies.
- Poor session management.
- Predictable session identifiers.
- Missing multi-factor authentication where appropriate.
- Insecure password-reset mechanisms.
- Unlimited login attempts.

Impact

- Unauthorized account access.
- Account takeover.
- Exposure of personal information.
- Unauthorized transactions or actions.
- Access to administrative functionality.

Prevention

Strong Password Protection

Passwords should be stored using strong, modern password-hashing mechanisms rather than plaintext or reversible encryption.

Multi-Factor Authentication

MFA provides an additional authentication factor beyond the password.

Secure Session Management

Applications should:

- Generate unpredictable session identifiers.
- Use HTTPS.
- Expire sessions appropriately.
- Invalidate sessions after logout.
- Protect session cookies with appropriate security attributes.

4) Security Misconfiguration

Security Misconfiguration occurs when a system, application, server, cloud service, or security feature is configured incorrectly or left unnecessarily exposed.

In simple words:

Secure software + wrong configuration = security risk

Common Examples

- Default usernames and passwords are still enabled.
- Unnecessary services are running.
- Debug mode is enabled in production.
- Detailed error messages reveal sensitive information.
- Security headers are missing or incorrectly configured.
- Software or frameworks are not securely configured.
- Cloud storage or other resources are accidentally made public.

Example

Suppose a production website displays a detailed error message containing:

- Database information.
- Internal file paths.
- Framework details.
- Configuration information.

An attacker could use this information to better understand the system.

Impact

- Information disclosure.
- Unauthorized access.
- Exposure of sensitive files.
- Increased attack surface.
- Application or server compromise.

Prevention

1. Remove Default Credentials

Change or disable default usernames and passwords before deployment.

2. Disable Debug Mode

Debugging features should normally be disabled in production environments.

3. Minimize Services

Run only the services and features that are actually required.

4. Secure Error Handling

Users should receive generic error messages rather than sensitive technical details.

5. Keep Systems Updated

Regularly update operating systems, frameworks, libraries, and other components.

Quick Revision Table

Topic	Simple Meaning	Main Risk	Main Prevention
SQL Injection	Malicious input affects database queries	Database/data compromise	Parameterized queries
XSS	Untrusted content executes in a browser	User/session compromise	Output encoding + CSP
Broken Authentication	Weak login/session controls	Account takeover	MFA + secure sessions
Security Misconfiguration	Incorrect/insecure system settings	Information exposure/unauthorized access	Secure configuration + regular review

Task 2: Install & Setup Tools

install **DVWA** and **OWASP ZAP** directly in Kali Linux.

1) Install DVWA dependencies

Cmd: `sudo apt install apache2 mariadb-server php php-mysqli php-gd git -y`

Start Apache and MariaDB cmd: `sudo systemctl start apache2`

`sudo systemctl start mariadb`

Enable them at startup cmd: `sudo systemctl enable apache2`

`sudo systemctl enable mariadb`

Check cmd: `sudo systemctl status apache2`

`sudo systemctl status mariadb`

You should see **active (running)**.

1.1 Download DVWA

Go to /var/www/html cmd: `cd /var/www/html`

Download DVWA cmd: `sudo git clone https://github.com/digininja/DVWA.git`

Give Apache ownership:

`sudo chown -R www-data:www-data /var/www/html/dvwa`

4. Configure DVWA

Go into the configuration directory cmd: `cd /var/www/html/dvwa/config`

Copy the configuration file: `sudo cp config.inc.php.dist config.inc.php`

Edit it: `sudo nano config.inc.php`

Find the database settings. You will see something similar to:

```
$_DVWA[ 'db_server' ] = '127.0.0.1';
```

```
$_DVWA[ 'db_database' ] = 'dvwa';
```

```
$_DVWA[ 'db_user' ] = 'dvwa';
```

```
$_DVWA[ 'db_password' ] = 'p@ssw0rd';
```

You can use these values for your local lab.

Save: **Ctrl + O** → **Enter** → **Ctrl + X**

5. Create the DVWA database

Open MariaDB cmd :`sudo mariadb`

Run:

```
CREATE DATABASE dvwa;
```

```
CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'p@ssw0rd';
```

```
GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';
```

```
FLUSH PRIVILEGES;
```

```
EXIT;
```

Restart Apache cmd: `sudo systemctl restart apache2`

6. Open DVWA

In Kali Firefox, go to:

`http://127.0.0.1/dvwa/`

You should see the DVWA page.

Then log in using the default DVWA credentials:

Username: admin

Password: password

Click: **Create / Reset Database**

7. Set DVWA Security Level

After logging in:

DVWA Security → **Security Level**

For your first experiments, select: Low

This is useful for understanding how the vulnerabilities work.

2) Install OWASP ZAP

Install command: sudo apt update
 sudo apt install zaproxy -y

Then cmd: zaproxy

You can also find it through:

Applications → **Web Application Analysis** → **OWASP ZAP**

Configure ZAP for DVWA

Once ZAP opens:

1. Start ZAP.
2. Choose the default session option.
3. Open the **Quick Start** tab.
4. Find **Automated Scan**.
5. Enter:

http://127.0.0.1/dvwa/

6. Start the scan.

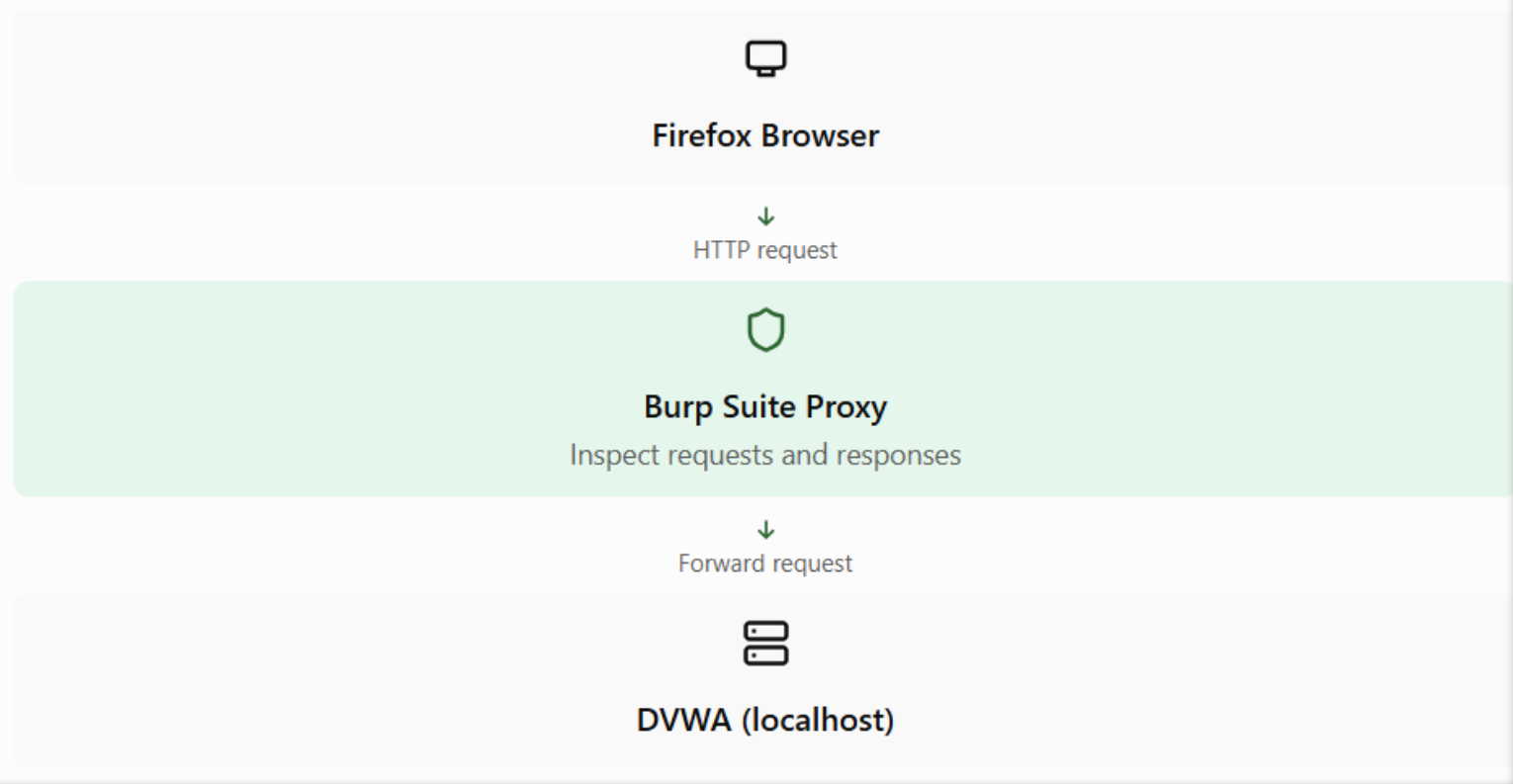
Because DVWA is running locally, the traffic stays inside your Kali lab

Task 3: Perform Testing (DVWA)

Configure Burp Suite Community Edition

Burp Suite acts as a proxy between your browser and DVWA. It allows you to inspect the information sent by the browser and the response received from the application.

How it works



3.1 SQL Injection Testing

Open the SQL Injection module

In DVWA's left-side menu, click: SQL Injection

Test a basic SQL Injection condition

In the same local DVWA input field, enter this educational test string:

1' OR '1'='1 Click Submit.

[Home](#)[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)[CSP Bypass](#)[JavaScript Attacks](#)[Authorisation Bypass](#)[Open HTTP Redirect](#)[Cryptography](#)[API](#)[DVWA Security](#)

Vulnerability: SQL Injection

User ID:

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

Capture the SQL Injection request in Burp

- 1) Return to Burp Suite.
- 2) Open Proxy → Intercept.
- 3) Turn Intercept on.
- 4) Submit the SQL Injection test in DVWA.
- 5) Switch to Burp.
- 6) Examine the captured request.
- 7) Take a screenshot of the request.
- 8) Click Forward.
- 9) Open Proxy → HTTP history.
- 10) Select the request and examine its response.

DashboardTargetProxyIntruderRepeaterCollaboratorSequencerDecoderComparerLoggerOrganizerExtensionsDiscover

InterceptHTTP historyWebSockets historyMatch and replaceProxy settings

Filter settings: Hiding CSS and image content; hiding specific extensionsPro version

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies
4	http://127.0.0.1	GET	/dvwa/vulnerabilities/sqli/index.php?id=		✓	200	5026	HTML	php	Vulnerability: SQL Injecti...			127.0.0.1	

Request

PrettyRawHex

1 GET /dvwa/vulnerabilities/sqli/index.php?id=1%27+or+%271%27%30%271&Submit=Submit&user_token=a2fd1286721f50493609f59130bbc99f HTTP/1.1
2 Host: 127.0.0.1
3 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Linux"
6 Accept-Language: en-US,en;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Referer: http://127.0.0.1/dvwa/vulnerabilities/sqli/index.php
15 Accept-Encoding: gzip, deflate, br
16 Cookie: security=impossible; PHPSESSID=c8b37701c2ec8a91d484b92805c3a447
17 Connection: keep-alive

Response

PrettyRawHexRender

1 HTTP/1.1 200 OK
2 Date: Tue, 29 Sep 2026 10:10:44 GMT
3 Server: Apache/2.4.68 (Debian)
4 Expires: Tue, 23 Jun 2009 12:00:00 GMT
5 Cache-Control: no-cache, must-revalidate
6 Pragma: no-cache
7 Vary: Accept-Encoding
8 Content-Length: 4698
9 Keep-Alive: timeout=5, max=100
10 Connection: Keep-Alive
11 Content-Type: text/html; charset=utf-8
12
13 <!DOCTYPE html>
14
15 <html lang="en-GB">
16
17 <head>
18 <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
19
20 <title>
21 Vulnerability: SQL Injection :: Damn Vulnerable Web Application (DVWA)
22 </title>
23
24 <link rel="stylesheet" type="text/css" href="../../dvwa/css/main.css" />
25
26 <link rel="icon" type="image/ico" href="../../favicon.ico" />
27
28 <script type="text/javascript" src="../../dvwa/js/dvwaPage.js">
29 </script>
30
31 </head>
32
33 <body class="home light">
34 <div id="container">
35 <div id="header">
36
37

Finding: SQL Injection

Testing observations:

- Normal input returned the expected database record.
- A specially crafted test input changed the behavior of the database query.
- The application returned results that differed from the normal query.
- Burp Suite successfully captured the HTTP request and response.

Mitigation methods:

1. Use parameterized queries or prepared statements.
2. Validate user input on the server side.
3. Avoid constructing SQL queries through direct string concatenation.

3.2 Reflected XSS Testing

Open the module In DVWA, click: XSS (Reflected)

You should see a page with a name input field.

[Home](#)[Instructions](#)[Setup / Reset DB](#)[Brute Force](#)[Command Injection](#)[CSRF](#)[File Inclusion](#)[File Upload](#)[Insecure CAPTCHA](#)[SQL Injection](#)[SQL Injection \(Blind\)](#)[Weak Session IDs](#)[XSS \(DOM\)](#)[XSS \(Reflected\)](#)[XSS \(Stored\)](#)

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Hello <script>alert('Ajith DVWA Test')</script>

More Information

- <https://owasp.org/www-community/attacks/xss/>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

Capture the request in Burp

1. Turn on Intercept in Burp.
2. Submit the test in DVWA.
3. Inspect the captured request.
4. Observe the submitted parameter.
5. Forward the request.
6. Check the response in HTTP history.

The screenshot displays the Burp Suite interface with the 'HTTP History' tab selected. A table lists captured requests, with the third request (index 7) selected. The 'Request' pane shows the raw HTTP request, and the 'Response' pane shows the raw HTTP response.

Request:

```
1 GET /dvwa/vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28%27Ajith+DVWA+Test%27%29%3C%2Fscript%3E&
2 user_token=877bf68ea674b8f6b1d70b478d1c384e HTTP/1.1
3 Host: 127.0.0.1
4 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
5 sec-ch-ua-mobile: ?0
6 sec-ch-ua-platform: "Linux"
7 Accept-Language: en-US,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0
10 Safari/537.36
11 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,appli
12 cation/signed-exchange;v=b3;q=0.7
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-User: ?1
16 Sec-Fetch-Dest: document
17 Referer: http://127.0.0.1/dvwa/vulnerabilities/xss_r/?name=ajith&user_token=c41d7eec6a8da7ead24cfc3b7efde5e
18 Accept-Encoding: gzip, deflate, br
19 Cookie: security=impossible; PHPSESSID=c8b37701c2ec8a91d484b92805c3a447
20 Connection: keep-alive
```

Response:

The response is an HTML page titled 'Vulnerability: Reflected Cross Site Scripting'. It contains a form with the text 'Hello <script>alert('Ajith DVWA Test')</script>' displayed below it, indicating a successful XSS attack.

Finding 2: Reflected Cross-Site Scripting (XSS)

Testing observations:

- The application accepted the test input.
- The browser displayed the expected demonstration.
- The submitted input was reflected in the application's response.
- Burp Suite captured the request and response, helping identify how the input was processed.

Mitigation methods:

1. Apply context-aware output encoding.
2. Validate input on the server side.
3. Use secure templating systems that automatically escape untrusted content.
4. Implement an appropriate Content Security Policy (CSP).
5. Avoid inserting untrusted data into executable JavaScript or HTML contexts.

3.3 Stored XSS Testing

Open the module In DVWA, click: XSS (Stored)

You should see a guestbook or message form.

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

DVWA

Vulnerability: Stored Cross Site Scripting (XSS)

Name *

ajith

Message *

This is my Week 6 security test.

Sign Guestbook

Clear Guestbook

Name: ajith

Message: This is my Week 6 security test.

More Information

- <https://owasp.org/www-community/attacks/xss>
- <https://owasp.org/www-community/xss-filter-evasion-cheatsheet>
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <https://www.cgisecurity.com/xss-faq.html>
- <https://www.scriptalert1.com/>

Test normal input

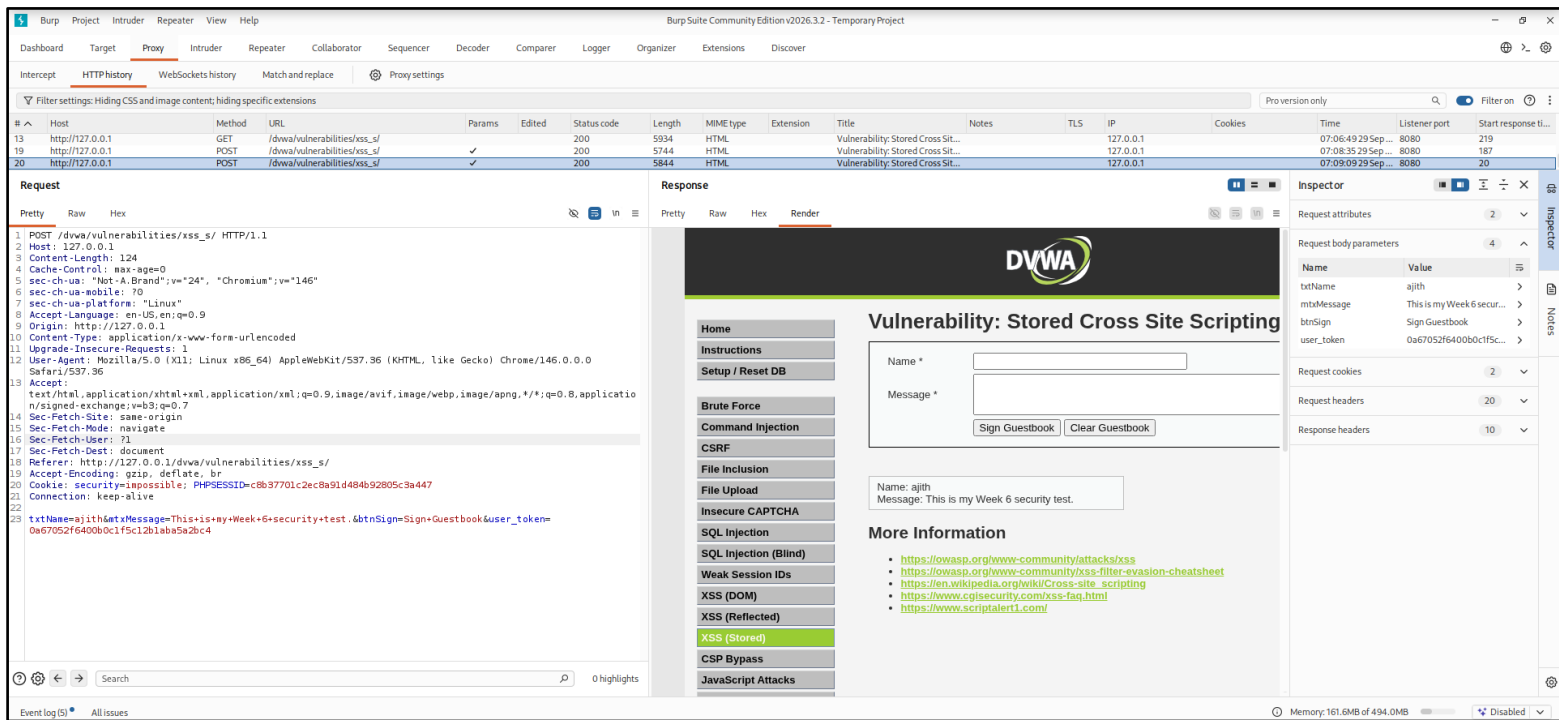
Enter:

Name: Ajith

Message: This is my Week 6 security test.

Capture the request in Burp

- 1. Enable Intercept.
- 2. Submit the guestbook message.
- 3. Inspect the POST request.
- 4. Observe the name and message parameters.
- 5. Forward the request.
- 6. Open HTTP history.
- 7. Inspect the server response.



Finding 3: Stored Cross-Site Scripting (XSS)

Testing observations:

- The application accepted and stored the test message.
- The submitted content remained available after refreshing the page.
- The browser interpreted the stored test input.
- Burp Suite captured the submission request and the corresponding response.

Mitigation methods:

- 1. Apply context-aware output encoding whenever stored content is displayed.
- 2. Validate and sanitize input according to its intended use.
- 3. Use safe templating frameworks.

- 4. Avoid rendering untrusted input as executable HTML or JavaScript.

Vulnerability Findings and Risk Levels

Summary of findings

SQL Injection High

User input influenced database query behavior, demonstrating insufficient input handling.

Reflected XSS Medium

Untrusted input was interpreted by the browser, demonstrating insufficient output encoding.

Stored XSS High

Untrusted content was stored and interpreted when displayed, potentially affecting multiple visitors.

Consolidated Vulnerability Assessment Table

This table is suitable for including in your report as the main findings summary.

Vulnerability	Evidence collected	Risk level	Recommended mitigation
SQL Injection	Modified database query results and Burp request/response	High	Parameterized queries, server-side validation, least-privilege database access
Reflected XSS	Browser demonstration and reflected input in response	Medium	Output encoding, safe templating, input validation, CSP
Stored XSS	Stored message and browser demonstration after submission	High	Context-aware output encoding, safe rendering, input validation, CSP

Task 4: Scan Using OWASP ZAP

Configure ZAP for DVWA

Once ZAP opens:

1. Start ZAP.

- Because DVWA is running locally, the traffic stays inside your Kali lab.



The following table provides a report format for documenting the findings from the scan. The exact alerts and risk levels should match the results in your exported ZAP report.

S. No.	Vulnerability / Alert	Risk Level	Description
1	Missing Anti-clickjacking Header	Medium	The application may not have appropriate protection against clickjacking attacks.
2	Content Security Policy (CSP) Header Not Set	Medium	The application does not provide a CSP header to help restrict the sources of browser-executable content.
3	X-Content-Type-Options Header Missing	Low	The application does not explicitly prevent browsers from interpreting content as a different MIME type.
4	Cookie Without HttpOnly Flag	Low	A cookie may be accessible to client-side scripts, increasing exposure if an XSS vulnerability exists.
5	Cookie Without SameSite Attribute	Low	The cookie may lack an additional browser control that helps reduce certain cross-site request risks.
6	Server Information Disclosure	Informational	Server response headers may reveal software or version information.

1) Missing Anti-clickjacking Header

Risk: Medium

Description:

The application does not provide an appropriate anti-clickjacking protection header, such as X-Frame-Options or a suitable CSP frame-ancestors directive.

Clickjacking is a technique in which a user is tricked into interacting with a webpage element that is embedded or disguised within another page.

Potential Impact:

- Users may be tricked into performing unintended actions.
- Sensitive application functions may be exposed to deceptive framing.
- User interaction may be manipulated.

Mitigation Methods:

- Configure the X-Frame-Options header where appropriate.
- Use the CSP frame-ancestors directive to control which websites can embed the application.
- Test the headers after implementation.

2) Content Security Policy (CSP) Header Not Set

Risk: Medium

Description:

The application does not define a Content Security Policy header. CSP is a browser security mechanism that helps control which sources are permitted to load scripts, styles, images, and other resources.

Potential Impact:

- Reduced protection against certain cross-site scripting attacks.
- Increased exposure to unwanted content loading.
- Fewer browser-level restrictions on potentially unsafe resources.

Mitigation Methods:

- Implement a suitable Content Security Policy.
- Restrict script sources to trusted locations.
- Avoid unsafe inline scripts where possible.
- Test the policy to ensure legitimate application functionality continues to work.

Conclusion & Learning

Week 6 – Conclusion

During Week 6 of my cybersecurity internship, I gained practical knowledge of web application security and vulnerability testing. I studied common OWASP vulnerabilities, including SQL Injection, Reflected XSS, Stored XSS, and Security Misconfiguration. Using DVWA, Burp Suite, and OWASP ZAP in Kali Linux, I performed vulnerability testing, captured HTTP requests and responses, and analyzed security findings. I learned how to identify risk levels, understand potential impacts, and recommend mitigation methods. This experience improved my understanding of VAPT tools, secure coding practices, and the importance of regular security assessments in protecting web applications.